Lab W09: The Loot Ledger

CIS 3300, Intermediate Object-Oriented Programming

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 The Story

The party is back from the Coppertop Mine, and every member came home with pockets full of loot. Each member reported their haul to the guild treasurer, who stored it all in a loot file.

The guild wants the loot stored in its vault. In the vault, identical items are kept together in one stack (three healing potions are one entry with a quantity of three, not three separate entries). When everything is stored, the vault is saved to a byte file so the guild can load it later. Your job is to build the vault.

2 What You Are Building

- **An `Item` class** that represents one kind of loot.
- **A `Vault` class** that holds the guild's loot. When loot is added and the vault already holds the same kind of item, the two are kept together as one stack instead of becoming separate entries.
- **A program with a `main` method** that reads the loot file the way we did in lecture, stores every piece of loot in a `Vault`, prints the vault's contents, and saves the vault to a byte file the way we did in lecture.

Store each piece of data in your classes using a data type that reasonably fits what it represents. In the byte file, each value must be written as the same data type your classes use to store it. For example, a value your class stores as an `int` is written to the file as an `int`.

You may add any other fields, methods, or helper classes you find useful.

3 Many Decisions Are Yours

This assignment deliberately does not tell you how to design your classes or when two pieces of loot are the same kind of item. You decide, and you will need to be able to explain why. This does not mean that "anything goes" for points.

There is more than one good answer, and your vault does not need to match a classmate's. What matters is that your decisions are deliberate, consistent with each other, and match what your code actually does.

Your decisions and your code must be consistent with what we have covered through Week 8, in both the lectures and the assigned book chapters; this assignment is about applying that knowledge. Inheritance and polymorphism are not part of this assignment, and your design is not expected to use them.

4 Explaining Your Decisions

At the top of `Item.java`, below the required comment header, explain every major design decision you made across all of your classes, in your own words. Be thorough, but keep each one to a line or two, not paragraphs. Selected students may be asked to walk through these decisions in a personal review, as described in the syllabus.

5 Project Setup

1. Create a new Java project named `labw09`.
2. Set up anything the project needs to read the loot file, the same way we did in lecture.
3. Put your classes in packages: `Item` and `Vault` in a package named `loot`, and your program with `main` in a package named `app`.
4. Create a `files` folder in the project's root folder and place the `loot` file in it. Save your byte file to that same folder.

6 Additional Sample Data

Three more loot files, `loot2`, `loot3`, and `loot4`, each larger than the last, are provided for testing. Place them in your `files` folder; your `Item` and `Vault` classes should work with any of them without changes.

7 Requirements Checklist

- Your classes are well designed, using encapsulation and the other principles of good class design we discussed in class.
- Your `Vault` keeps loot organized, with the same kind of item kept together in one stack.
- Your program reads the loot file the way we did in lecture.
- Your program prints the vault's contents in a form a person can read.
- Your program saves the vault to a byte file the way we did in lecture, writing each value as the data type your classes use to store it.
- Your decisions comment covers every major design decision, and your code matches what it says.
- Your code meets the course's standards for quality and commenting, including the syllabus's Code Commenting Requirements.

8 Grading

Items in Tables 1 and 2 are worth 1 point each and are either met or not met. Items in Table 3 are worth a range of points, with partial credit based on how well your work meets the standards we set in lecture. Because there is more than one good design for this assignment, grading of the Table 3 items includes some subjectivity.

Table 1: Submission and setup (1 point each)

Item	Points
Submitted as a reasonable attempt	1
`Item.java` and `Vault.java` compile and run in Eclipse without errors, declared in the `loot` package	1
The program with `main` compiles and runs in Eclipse without errors, declared in the `app` package	1
Project pushed to GitHub	1

Table 2: Code quality (1 point each)

Item	Points
Consistent naming conventions used throughout	1
Consistent formatting and indentation throughout	1
No unrequested extra output	1
`Item.java` has required documentation	1
`Vault.java` has required documentation	1
The program with `main` has required documentation	1
Loop and method logic reads clearly from top to bottom	1

Table 3: Design and implementation (partial credit possible)

Item	Points
The `Item` class meets expectations	10
The `Vault` class meets expectations	6
The loot file is read using the approach from lecture	5
The vault is saved using the approach from lecture	5
Design decisions are explained, and the code matches the explanation	8

Total: 45 points.

9 Submission

Submit through your course GitHub repository, per the syllabus's submission guidelines. Include your `src` folder with the `loot` and `app` packages, and the `files` folder with the `loot` file. It is fine either way whether your byte file is included, since your program creates it every time it runs.